Tunneling v5

Implementation code - supervised tunnel, agent bus, telemetry

BRIDGE 5.0.0

BRANCH	arena/706ae6d5-tunneling
DIFF BASE	c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524 -> HEAD
GENERATED	2026-11-03
TESTS	25 passing (python3 -B roblox-bridge/test_bridge_mcp.py)
RUNTIME	Python 3 stdlib only (no pip packages)

INSIDE

1 What this release contains · 2 File inventory · 3 How to apply and verify · 4 Complete sources (tunnel_manager, bus_core, auth_core, telemetry, seed_agent_bus) · 5 Unified diffs for every modified file · 6 Verification snapshot

1. What this release contains

v5 turns the local Studio bridge into an ops platform. The MCP tool surface grows to **70 tools** (adds `agent_bus` and `bridge_ops`) and **8 resources**. Four new stdlib modules carry the new behaviour, and the existing server/CLI/setup paths are extended without breaking v4 callers.

- **Supervised tunnel** - auto-detected cloudflared/ngrok, URL scraping, public health probes, backoff restarts, named tunnels, LAN URLs, and a hard refusal to publish a token-less bridge.
- **Agent Bus** - presence + mailbox + lease-based task board so multiple agents and humans can share one Studio session without collisions.
- **Scoped tokens** - hashed read/write/admin credentials with expiry and per-token rate limits; read tokens cannot mutate Studio over MCP or REST.
- **Resilient jobs** - idempotency keys, priorities, bounded retries, per-job timeouts and orphan requeue.
- **Observability** - Prometheus metrics, SSE event stream, SHA-256 hash-chained audit log, consolidated ops report, and dashboard tab 7.

2. File inventory

File	Kind	Purpose
`roblox-bridge/tunnel_manager.py`	new	Supervised cloudflared/ngrok tunnel
`roblox-bridge/bus_core.py`	new	Multi-agent presence, mailbox and lease-based task board
`roblox-bridge/auth_core.py`	new	Scoped tokens, expiry, revocation, token-bucket limits
`roblox-bridge/telemetry.py`	new	Metrics, SSE event stream, hash-chained audit log
`roblox-bridge/tools/seed_agent_bus.py`	new	Idempotent Agent Bus seeding (online or offline)
`roblox-bridge/server.py`	diff	HTTP bridge, auth scopes, ops endpoints, dashboard tab 7
`roblox-bridge/mcp_core.py`	diff	agent_bus + bridge_ops tools, 3 resources, client helpers
`roblox-bridge/arena_studio.py`	diff	ops / metrics / audit / tunnel / tokens / bus CLI
`roblox-bridge/setup.py`	diff	Supervised tunnel launch + URL watcher
`roblox-bridge/test_bridge_mcp.py`	diff	v5 test coverage (scopes, bus, reaping, audit, tunnel)
`roblox-bridge/mcp_server.py`	diff	Version banner
`roblox-bridge/ab.ps1`	diff	Version banner + ops examples

3. How to apply and verify

Copy the new files into `roblox-bridge/` and apply the diffs to the modified files (both sides are shown in this document). Then:

```
python3 -B roblox-bridge/test_bridge_mcp.py # expect: Ran 25 tests ... OK
python3 roblox-bridge/arena_studio.py ops # peers, queue, tunnel, agents, tasks
python3 roblox-bridge/arena_studio.py audit --verify # hash chain intact
python3 setup.py --tunnel # supervised public tunnel
```

Auth is unchanged for existing installs: `bridge.token` is the admin owner credential and nothing new is required. Runtime state (audit, tokens, bus, last tunnel URL) is written to the gitignored `roblox-bridge/bridge_state/`.

4. New source files

File	Lines	Purpose
`roblox-bridge/tunnel_manager.py`	475	Supervised cloudflared/ngrok tunnel
`roblox-bridge/bus_core.py`	752	Multi-agent presence, mailbox and lease-based task board
`roblox-bridge/auth_core.py`	273	Scoped tokens, expiry, revocation, token-bucket limits
`roblox-bridge/telemetry.py`	471	Metrics, SSE event stream, hash-chained audit log
`roblox-bridge/tools/seed_agent_bus.py`	129	Idempotent Agent Bus seeding (online or offline)

Complete sources follow first, then unified diffs for every modified file. Line numbers are left-margin references; wrapped lines carry no number.

roblox-bridge/tunnel_manager.py

475 lines

Supervised cloudflared/ngrok tunnel

```python
#!/usr/bin/env python3
"""
tunnel_manager.py - Supervised public tunnel for the Arena Roblox Studio bridge.

`python setup.py --tunnel` starts a one-shot Cloudflare quick tunnel and hopes it
stays up. This module makes the tunnel a managed service instead:

  * auto-detects `cloudflared` or `ngrok` on PATH (with install hints if absent),
  * supervises the child process with exponential backoff and automatic restart,
  * scrapes the public URL from the child's output and re-learns it after a restart,
  * probes `<publicUrl>/api/health` and marks the tunnel degraded when it stops
    answering, restarting it after repeated failures,
  * supports token-based named Cloudflare tunnels and a user-managed static URL,
  * reports LAN URLs so phones/tablets on the same Wi-Fi can reach the dashboard,
  * exposes everything as JSON for `GET /api/tunnel` and the Control Center.

Zero dependencies; works on Windows, macOS and Linux.
"""

import json
import os
import re
import shutil
import socket
import subprocess
import threading
import time
import urllib.error
import urllib.request
from collections import deque
from pathlib import Path

TUNNEL_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

CLOUDFLARE_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9-]*\.trycloudflare\.com", re.I)
NGROK_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9-.]*\.ngrok(?:-free)?\.(?:app|io|dev)", re.I)
ANY_URL_RE = re.compile(r"https://[a-z0-9][a-z0-9.-]+\.[a-z]{2,}(?::\d+)?", re.I)

INSTALL_HINTS = {
    "cloudflared": [
        "Windows:  winget install --id Cloudflare.cloudflared",
        "macOS:    brew install cloudflared",
        "Linux:    https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/",
    ],
    "ngrok": [
        "Windows:  winget install --id ngrok.ngrok",
        "macOS:    brew install ngrok/ngrok/ngrok",
        "Any:      https://ngrok.com/download (then `ngrok config add-authtoken <token>`)",
    ],
}


def _now() -> float:
    return time.time()


def local_ip_addresses() -> list:
    """Best-effort LAN IPv4 addresses (no packet is actually sent)."""
    found = []
```

```
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            sock.connect(("10.255.255.255", 1))
            found.append(sock.getsockname()[0])
        finally:
            sock.close()
    except OSError:
        pass
    try:
        host = socket.gethostname()
        for info in socket.getaddrinfo(host, None, socket.AF_INET):
            ip = info[4][0]
            if ip not in found and not ip.startswith("127."):
                found.append(ip)
    except OSError:
        pass
    return [ip for ip in found if not ip.startswith("127.")]


def find_tunnel_binary(preferred: str = None) -> tuple:
    """Return (flavor, path) for the first tunnel binary found, else (None, None)."""
    candidates = []
    if preferred and preferred not in ("auto", ""):
        candidates.append(preferred)
    candidates += ["cloudflared", "ngrok"]
    for name in candidates:
        found = shutil.which(name)
        if found:
            return ("cloudflared" if "cloudflared" in Path(found).name.lower() else "ngrok"), found
    return None, None


class TunnelManager:
    """Starts, watches, restarts and reports a public tunnel for the local bridge."""

    def __init__(
        self,
        port: int = 8077,
        binary: str = None,
        flavor: str = "auto",
        named_token: str = None,
        name: str = None,
        public_url: str = None,
        health_interval: float = 20.0,
        health_failures_before_restart: int = 4,
        max_backoff: float = 30.0,
        state_dir=None,
        auto_start: bool = False,
    ):
        self.port = int(port)
        self.local_url = f"http://127.0.0.1:{self.port}"
        self.requested_flavor = (flavor or "auto").lower()
        self.binary_arg = binary
        self.named_token = named_token or os.environ.get("TUNNEL_TOKEN") or None
        self.name = name
        self.user_public_url = (public_url or os.environ.get("BRIDGE_PUBLIC_URL") or "").strip() or None
        self.health_interval = float(health_interval)
        self.health_failures_before_restart = int(health_failures_before_restart)
        self.max_backoff = float(max_backoff)
```

```
        self.state_path = (Path(state_dir) if state_dir else DEFAULT_STATE_DIR) / "tunnel.json"

        self.lock = threading.RLock()
        self.stop_event = threading.Event()
        self._thread = None
        self._proc = None
        self.flavor = None
        self.binary = None
        self.state = "stopped"
        self.public_url = self.user_public_url
        self.last_error = None
        self.started_at = None
        self.online_at = None
        self.restarts = 0
        self.url_changes = 0
        self.url_changed_at = None
        self.proc_pid = None
        self.health = {"lastProbe": None, "ok": None, "latencyMs": None, "failures": 0, "consecutiveFailures": 0}
        self.log_tail = deque(maxlen=60)
        self.last_output_at = None
        self._last_health = 0.0
        self._load_last_url()
        if auto_start:
            self.start()

    # -- persistence of last known URL ---------------------------------------

    def _load_last_url(self):
        try:
            raw = json.loads(self.state_path.read_text(encoding="utf-8"))
            if not self.public_url and raw.get("publicUrl"):
                self.public_url = raw["publicUrl"]
                self.last_error = None
                self._loaded_url = True
        except (OSError, ValueError):
            pass

    def _persist_url(self):
        try:
            self.state_path.parent.mkdir(parents=True, exist_ok=True)
            self.state_path.write_text(
                json.dumps({"publicUrl": self.public_url, "saved": _now(), "version": TUNNEL_VERSION}),
                encoding="utf-8",
            )
        except OSError:
            pass

    # -- command construction ------------------------------------------------

    def _command(self) -> list:
        if not self.binary:
            raise RuntimeError("No tunnel binary available")
        if self.flavor == "cloudflared":
            if self.named_token:
                # Named tunnel: the ingress/hostname config lives in Cloudflare;
                # `--url` is only valid for quick tunnels.
                return [self.binary, "tunnel", "--no-autoupdate", "run", "--token", self.named_token]
            return [self.binary, "tunnel", "--url", self.local_url, "--no-autoupdate"]
        if self.flavor == "ngrok":
            cmd = [self.binary, "http", str(self.port), "--log", "stdout", "--log-format", "logfmt"]
```

```
            return cmd
        raise RuntimeError(f"Unsupported tunnel flavor: {self.flavor}")

    def _detect(self):
        flavor, path = find_tunnel_binary(self.binary_arg)
        if not flavor and self.requested_flavor in ("cloudflared", "ngrok"):
            # Respect an explicit preference even if the generic detection failed.
            found = shutil.which(self.requested_flavor)
            if found:
                flavor, path = self.requested_flavor, found
        with self.lock:
            self.flavor = flavor if not self.requested_flavor or self.requested_flavor == "auto" else (self.requested_flavor if path else flavor)
            self.binary = path
        return bool(path)

    def available(self) -> bool:
        return self.binary is not None or shutil.which(self.binary_arg or "") is not None

    # -- lifecycle ---------------------------------------------------------

    def start(self) -> dict:
        with self.lock:
            if self.user_public_url:
                self.state = "online"
                self.public_url = self.user_public_url
                self.online_at = self.online_at or _now()
                return self.status()
            if self._thread and self._thread.is_alive():
                return self.status()
            if not self._detect():
                self.state = "unavailable"
                self.last_error = (
                    "No tunnel binary found on PATH. Install cloudflared or ngrok, "
                    "or pass BRIDGE_PUBLIC_URL for a tunnel you run yourself."
                )
                return self.status()
            self.stop_event.clear()
            self.last_error = None
            self.state = "starting"
            self.started_at = self.started_at or _now()
            self._thread = threading.Thread(target=self._supervise, name="tunnel-supervisor", daemon=True)
            self._thread.start()
            return self.status()

    def stop(self) -> dict:
        self.stop_event.set()
        with self.lock:
            proc = self._proc
        if proc and proc.poll() is None:
            try:
                proc.terminate()
                try:
                    proc.wait(timeout=5)
                except subprocess.TimeoutExpired:
                    proc.kill()
            except OSError:
                pass
        thread = self._thread
        if thread and thread.is_alive() and thread is not threading.current_thread():
            thread.join(timeout=6)
```

```
        with self.lock:
            self._thread = None
            self._proc = None
            self.proc_pid = None
            self.state = "stopped"
            if not self.user_public_url:
                self.online_at = None
        return self.status()

    def restart(self) -> dict:
        self.stop()
        with self.lock:
            self.restarts += 1
        time.sleep(0.4)
        return self.start()

    # -- supervision -------------------------------------------------------

    def _supervise(self):
        backoff = 1.0
        while not self.stop_event.is_set():
            try:
                cmd = self._command()
            except RuntimeError as exc:
                with self.lock:
                    self.state = "error"
                    self.last_error = str(exc)
                return
            with self.lock:
                self.state = "starting"
                self._append_log_unlocked(f"$ {' '.join(cmd[:3])}{' ...' if len(cmd) > 3 else ''}")
            try:
                proc = subprocess.Popen(
                    cmd,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.STDOUT,
                    text=True,
                    bufsize=1,
                    start_new_session=(os.name != "nt"),
                )
            except (OSError, ValueError) as exc:
                with self.lock:
                    self.state = "error"
                    self.last_error = f"Failed to launch {self.flavor}: {exc}"
                    self._append_log_unlocked(self.last_error)
                if self.stop_event.wait(backoff):
                    break
                backoff = min(self.max_backoff, backoff * 2)
                continue
            with self.lock:
                self._proc = proc
                self.proc_pid = proc.pid
            reader = threading.Thread(target=self._read_output, args=(proc,), daemon=True)
            reader.start()
            # Wait for the process to exit, probing health in the meantime.
            while not self.stop_event.is_set() and proc.poll() is None:
                if self.stop_event.wait(timeout=1.0):
                    break
                self._maybe_probe()
            reader.join(timeout=2)
```

```
            if self.stop_event.is_set():
                break
            code = proc.poll()
            with self.lock:
                self._proc = None
                self.proc_pid = None
                self._append_log_unlocked(f"[supervisor] {self.flavor} exited with code {code}; reconnecting...")
                self.state = "reconnecting"
                self.restarts += 1
                self.health["ok"] = False
                self.health["consecutiveFailures"] = 0
                # A dead child invalidates its quick-tunnel URL; a new one is
                # scraped from the replacement process.
                self.public_url = None
            if self.stop_event.wait(backoff):
                break
            backoff = min(self.max_backoff, backoff * 2)
        with self.lock:
            if self.state not in ("stopped",):
                self.state = "stopped"

    def _read_output(self, proc):
        stream = proc.stdout
        if not stream:
            return
        for raw_line in iter(stream.readline, ""):
            line = (raw_line or "").rstrip()
            if not line:
                continue
            with self.lock:
                self._append_log_unlocked(line)
                self.last_output_at = _now()
            match = None
            if self.flavor == "ngrok":
                match = NGROK_URL_RE.search(line) or CLOUDFLARE_URL_RE.search(line)
            else:
                match = CLOUDFLARE_URL_RE.search(line) or NGROK_URL_RE.search(line)
            if match:
                self._set_url(match.group(0))
        try:
            stream.close()
        except OSError:
            pass

    def _append_log_unlocked(self, line: str):
        self.log_tail.append(line[:400])

    def _set_url(self, url: str):
        url = url.rstrip("/")
        with self.lock:
            if self.public_url == url:
                return
            changed = self.public_url is not None
            self.public_url = url
            self.url_changed_at = _now()
            self.state = "online"
            self.online_at = _now()
            self.last_error = None
            self.health["ok"] = None
            self.health["failures"] = 0
```

```
            self.health["consecutiveFailures"] = 0
            if changed:
                self.url_changes += 1
        self._persist_url()

    def _maybe_probe(self):
        now = _now()
        if now - self._last_health < self.health_interval:
            return
        if not self.public_url:
            self._last_health = now
            if self.last_output_at and now - self.last_output_at > 45 and self.state == "starting":
                with self.lock:
                    self.last_error = "Tunnel process started but has not published a URL yet."
            return
        self._last_health = now
        threading.Thread(target=self._probe_once, name="tunnel-health", daemon=True).start()

    def _probe_once(self):
        url = f"{self.public_url}/api/health"
        started = time.monotonic()
        ok = False
        detail = None
        try:
            req = urllib.request.Request(url, headers={"User-Agent": f"arena-tunnel/{TUNNEL_VERSION}"})
            with urllib.request.urlopen(req, timeout=10) as resp:
                ok = 200 <= resp.status < 500
                detail = f"HTTP {resp.status}"
        except urllib.error.HTTPError as exc:
            ok = exc.code < 500
            detail = f"HTTP {exc.code}"
        except Exception as exc:  # noqa: BLE001 - network errors are all non-fatal
            detail = str(exc)[:200]
        latency = round((time.monotonic() - started) * 1000, 1)
        restart = False
        with self.lock:
            self.health["lastProbe"] = _now()
            self.health["latencyMs"] = latency
            self.health["ok"] = ok
            self.health["detail"] = detail
            if ok:
                self.health["failures"] = 0
                self.health["consecutiveFailures"] = 0
                if self.state == "degraded":
                    self.state = "online"
            else:
                self.health["failures"] = int(self.health.get("failures") or 0) + 1
                self.health["consecutiveFailures"] = int(self.health.get("consecutiveFailures") or 0) + 1
                if self.health["consecutiveFailures"] >= 2 and self.state == "online":
                    self.state = "degraded"
                if self.health["consecutiveFailures"] >= self.health_failures_before_restart:
                    restart = True
            if restart:
                self._append_log_unlocked("[supervisor] public URL failed repeated health probes; restarting tunnel")
        if restart:
            proc = self._proc
            if proc and proc.poll() is None:
                try:
                    proc.terminate()
                except OSError:
```

```
                    pass
            with self.lock:
                self.restarts += 1

    # -- reporting ------------------------------------------------------------

    def status(self) -> dict:
        with self.lock:
            now = _now()
            state = self.state
            if self.user_public_url and state == "stopped":
                state = "online"
            uptime = round(now - self.online_at, 1) if self.online_at and state == "online" else 0
            lan = [
                f"http://{ip}:{self.port}" for ip in local_ip_addresses()
            ]
            return {
                "version": TUNNEL_VERSION,
                "state": state,
                "running": bool(self._thread and self._thread.is_alive()) or bool(self.user_public_url),
                "flavor": self.flavor,
                "binary": self.binary,
                "binaryAvailable": bool(self.binary),
                "managed": not bool(self.user_public_url),
                "named": bool(self.named_token),
                "localUrl": self.local_url,
                "lanUrls": lan,
                "publicUrl": self.public_url,
                "mcpEndpoint": f"{self.public_url}/mcp" if self.public_url else None,
                "healthEndpoint": f"{self.public_url}/api/health" if self.public_url else None,
                "onlineSeconds": uptime,
                "restarts": self.restarts,
                "urlChanges": self.url_changes,
                "urlChangedAt": self.url_changed_at,
                "pid": self.proc_pid,
                "lastError": self.last_error,
                "health": dict(self.health),
                "logTail": list(self.log_tail)[-25:],
                "installHints": INSTALL_HINTS.get(self.flavor or "cloudflared"),
                "serverTime": now,
            }

    def handle(self, payload: dict) -> dict:
        action = ((payload or {}).get("action") or "status").lower()
        if action == "status":
            return self.status()
        if action == "start":
            return self.start()
        if action == "stop":
            return self.stop()
        if action == "restart":
            return self.restart()
        if action == "lan":
            return {"lanUrls": [f"http://{ip}:{self.port}" for ip in local_ip_addresses()], "port": self.port}
        raise ValueError("Unknown tunnel action. Use status, start, stop, restart or lan.")

```

roblox-bridge/bus_core.py

752 lines

Multi-agent presence, mailbox and lease-based task board

```python
#!/usr/bin/env python3
"""
bus_core.py - Multi-agent collaboration bus for the Arena Roblox Studio bridge.

Zero-dependency, thread-safe, JSON-persisted coordination layer that lets several
AI agents (and humans) share one Roblox Studio session without stepping on each
other:

  * Presence registry   - named agents with roles, capabilities and heartbeats.
  * Mailbox             - direct + broadcast messages, cursor-based inbox, acks.
  * Task board          - claim/lease/complete work items with priorities, tags
                          and dependencies. A lease expires automatically, so a
                          crashed agent cannot permanently hold a task.
  * Event feed          - bounded, sequence-numbered activity log.
  * Handoff export      - markdown brief for the next agent.

Everything is plain stdlib so it can be embedded in server.py, used from the CLI,
or driven through the `agent_bus` MCP tool.
"""

import json
import os
import threading
import time
import uuid
from collections import deque
from pathlib import Path

BUS_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

AGENT_STATUSES = ("online", "busy", "idle", "offline")
TASK_STATUSES = ("todo", "in_progress", "blocked", "done", "cancelled")
MESSAGE_KINDS = ("note", "question", "answer", "handoff", "alert", "result")
PRIORITIES = ("low", "normal", "high", "critical")
_PRIORITY_RANK = {"low": 3, "normal": 2, "high": 1, "critical": 0}


def _now() -> float:
    return time.time()


def _clean(text, limit: int = 8000) -> str:
    if text is None:
        return ""
    text = str(text)
    return text[:limit]


class AgentBus:
    """Presence + mailbox + task board shared by every agent on the bridge."""

    def __init__(
        self,
        path=None,
        max_messages: int = 2000,
        max_events: int = 1000,
        default_lease_seconds: int = 900,
        presence_ttl: float = 90.0,
        persist: bool = True,
```

```
    ):
        self.path = Path(path) if path else DEFAULT_STATE_DIR / "bus.json"
        self.persist = bool(persist)
        self.lock = threading.RLock()
        self.agents = {}
        self.messages = deque(maxlen=max_messages)
        self.events = deque(maxlen=max_events)
        self.tasks = {}
        self.default_lease_seconds = int(default_lease_seconds)
        self.presence_ttl = float(presence_ttl)
        self.seq = 0
        self.created = _now()
        self._load()

    # -- persistence -----------------------------------------------------

    def _load(self):
        if not self.persist or not self.path.is_file():
            return
        try:
            raw = json.loads(self.path.read_text(encoding="utf-8"))
        except (OSError, ValueError):
            return
        if not isinstance(raw, dict):
            return
        with self.lock:
            self.agents = {a["id"]: a for a in raw.get("agents", []) if a.get("id")}
            self.messages = deque(raw.get("messages", [])[-self.messages.maxlen:], maxlen=self.messages.maxlen)
            self.events = deque(raw.get("events", [])[-self.events.maxlen:], maxlen=self.events.maxlen)
            self.tasks = {t["id"]: t for t in raw.get("tasks", []) if t.get("id")}
            self.seq = int(raw.get("seq") or 0)

    def _save(self):
        if not self.persist:
            return
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            payload = {
                "version": BUS_VERSION,
                "saved": _now(),
                "seq": self.seq,
                "agents": list(self.agents.values()),
                "messages": list(self.messages),
                "events": list(self.events),
                "tasks": list(self.tasks.values()),
            }
            tmp = self.path.with_suffix(".tmp")
            tmp.write_text(json.dumps(payload, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
            os.replace(tmp, self.path)
        except OSError:
            pass

    # -- internals -------------------------------------------------------

    def _next_seq_unlocked(self) -> int:
        self.seq += 1
        return self.seq

    def _emit_unlocked(self, kind: str, data: dict) -> dict:
        event = {"seq": self._next_seq_unlocked(), "kind": kind, "ts": _now(), "data": data}
```

```
        self.events.append(event)
        return event

    def _agent_unlocked(self, agent_id: str) -> dict:
        agent = self.agents.get(agent_id)
        if not agent:
            raise KeyError(f"Unknown agent id: {agent_id!r}")
        return agent

    def _touch_unlocked(self, agent: dict):
        agent["lastSeen"] = _now()
        agent["online"] = True

    def _expire_leases_unlocked(self):
        now = _now()
        for task in self.tasks.values():
            lease = task.get("lease") or {}
            if task.get("status") == "in_progress" and lease.get("expiresAt") and lease["expiresAt"] < now:
                prev_owner = task.get("assignee")
                task["status"] = "todo"
                task["assignee"] = None
                task["lease"] = None
                task["updated"] = now
                task["leaseExpiries"] = int(task.get("leaseExpiries", 0)) + 1
                self._emit_unlocked("task.lease_expired", {"taskId": task["id"], "previousAssignee": prev_owner})

    # -- agents / presence ---------------------------------------------------

    def register_agent(
        self,
        name: str,
        role: str = "agent",
        capabilities=None,
        agent_id: str = None,
        ttl: float = None,
        meta: dict = None,
    ) -> dict:
        name = (name or "").strip()
        if not name:
            raise ValueError("name is required")
        with self.lock:
            # Reuse an existing registration with the same name (case-insensitive).
            existing = next((a for a in self.agents.values() if a["name"].lower() == name.lower()), None)
            now = _now()
            caps = [str(c) for c in (capabilities or [])][:32]
            if existing:
                existing.update({
                    "name": name,
                    "role": _clean(role, 40) or "agent",
                    "capabilities": caps or existing.get("capabilities", []),
                    "status": "online",
                    "online": True,
                    "lastSeen": now,
                    "ttl": float(ttl or self.presence_ttl),
                    "meta": meta if isinstance(meta, dict) else existing.get("meta", {}),
                })
                agent = existing
                created_new = False
            else:
                aid = agent_id or ("agent-" + uuid.uuid4().hex[:8])
```

```
                if aid in self.agents:
                    aid = "agent-" + uuid.uuid4().hex[:8]
                agent = {
                    "id": aid,
                    "name": name,
                    "role": _clean(role, 40) or "agent",
                    "capabilities": caps,
                    "status": "online",
                    "online": True,
                    "firstSeen": now,
                    "lastSeen": now,
                    "ttl": float(ttl or self.presence_ttl),
                    "meta": meta if isinstance(meta, dict) else {},
                }
                self.agents[agent["id"]] = agent
                created_new = True
            self._emit_unlocked("agent.register" if created_new else "agent.heartbeat", {
                "agentId": agent["id"], "name": agent["name"], "role": agent["role"],
            })
            self._save()
            return self._agent_view_unlocked(agent)

    def heartbeat(self, agent_id: str, status: str = None) -> dict:
        with self.lock:
            agent = self._agent_unlocked(agent_id)
            self._touch_unlocked(agent)
            if status in AGENT_STATUSES:
                agent["status"] = status
            self._save()
            return self._agent_view_unlocked(agent)

    def set_status(self, agent_id: str, status: str) -> dict:
        if status not in AGENT_STATUSES:
            raise ValueError(f"status must be one of {', '.join(AGENT_STATUSES)}")
        with self.lock:
            agent = self._agent_unlocked(agent_id)
            self._touch_unlocked(agent)
            agent["status"] = status
            self._save()
            return self._agent_view_unlocked(agent)

    def _agent_view_unlocked(self, agent: dict) -> dict:
        row = dict(agent)
        age = _now() - float(agent.get("lastSeen") or 0)
        row["ageSeconds"] = round(age, 1)
        row["online"] = age <= float(agent.get("ttl") or self.presence_ttl)
        if not row["online"]:
            row["status"] = "offline"
        row["openTasks"] = sum(
            1 for t in self.tasks.values()
            if t.get("assignee") == agent["id"] and t.get("status") in ("todo", "in_progress", "blocked")
        )
        row["inboxUnread"] = sum(
            1 for m in self.messages
            if (m.get("to") in (agent["id"], "all", None)) and agent["id"] not in (m.get("readBy") or {})
        )
        return row

    def list_agents(self, include_offline: bool = True) -> dict:
        with self.lock:
```

```
            rows = [self._agent_view_unlocked(a) for a in self.agents.values()]
        if not include_offline:
            rows = [r for r in rows if r["online"]]
        rows.sort(key=lambda r: (not r["online"], r["name"].lower()))
        return {"agents": rows, "count": len(rows), "onlineCount": sum(1 for r in rows if r["online"])}

    # -- mailbox ------------------------------------------------------------

    def post_message(
        self,
        sender: str,
        body: str,
        to: str = None,
        subject: str = "",
        kind: str = "note",
        priority: str = "normal",
        thread_id: str = None,
        meta: dict = None,
    ) -> dict:
        sender = (sender or "").strip()
        if not sender:
            raise ValueError("sender is required")
        if not (body or subject):
            raise ValueError("body or subject is required")
        if kind not in MESSAGE_KINDS:
            kind = "note"
        if priority not in PRIORITIES:
            priority = "normal"
        with self.lock:
            # Accept an agent name as well as an id; "all" / None means broadcast.
            to_id = None
            if to and to not in ("all", "*", "broadcast"):
                to_id = to
                if to_id not in self.agents:
                    match = next((a["id"] for a in self.agents.values() if a["name"].lower() == str(to).lower()), None)
                    to_id = match or to
            msg = {
                "id": "msg-" + uuid.uuid4().hex[:10],
                "seq": self._next_seq_unlocked(),
                "ts": _now(),
                "from": sender,
                "fromId": next((a["id"] for a in self.agents.values() if a["name"].lower() == sender.lower()), None),
                "to": to_id,
                "subject": _clean(subject, 300),
                "body": _clean(body),
                "kind": kind,
                "priority": priority,
                "threadId": thread_id,
                "meta": meta if isinstance(meta, dict) else {},
                "readBy": {},
            }
            self.messages.append(msg)
            self._emit_unlocked("message.post", {
                "id": msg["id"], "from": sender, "to": to_id or "broadcast",
                "subject": msg["subject"], "kind": kind, "priority": priority,
            })
            self._save()
            return msg

    def inbox(
```

```
        self,
        agent_id: str,
        since: int = 0,
        limit: int = 50,
        unread_only: bool = False,
        include_broadcast: bool = True,
    ) -> dict:
        limit = max(1, min(int(limit or 50), 500))
        with self.lock:
            agent = self.agents.get(agent_id)
            names = {agent_id}
            if agent:
                names.add(agent["name"].lower())
            rows = []
            for msg in reversed(self.messages):
                if since and msg["seq"] <= since:
                    break
                target = msg.get("to")
                if target is None:
                    if not include_broadcast:
                        continue
                elif target not in names and str(target).lower() not in names:
                    continue
                if msg.get("from") == agent_id:
                    continue
                if unread_only and agent_id in (msg.get("readBy") or {}):
                    continue
                rows.append(msg)
                if len(rows) >= limit:
                    break
            rows.reverse()
            unread = sum(
                1 for m in self.messages
                if (m.get("to") is None or m.get("to") in names or str(m.get("to", "")).lower() in names)
                and agent_id not in (m.get("readBy") or {}) and m.get("from") != agent_id
            )
            return {
                "agentId": agent_id,
                "messages": rows,
                "count": len(rows),
                "unread": unread,
                "cursor": rows[-1]["seq"] if rows else since,
            }

    def read_thread(self, thread_id: str) -> dict:
        with self.lock:
            rows = [m for m in self.messages if m.get("threadId") == thread_id or m["id"] == thread_id]
        return {"threadId": thread_id, "messages": rows, "count": len(rows)}

    def mark_read(self, agent_id: str, message_ids=None, all_messages: bool = False) -> dict:
        now = _now()
        marked = 0
        with self.lock:
            for msg in self.messages:
                if not all_messages and message_ids and msg["id"] not in message_ids:
                    continue
                if all_messages or message_ids:
                    if agent_id not in (msg.get("readBy") or {}):
                        msg.setdefault("readBy", {})[agent_id] = now
                        marked += 1
```

```
            self._save()
        return {"agentId": agent_id, "marked": marked}

    # -- task board -----------------------------------------------------------

    def create_task(
        self,
        title: str,
        detail: str = "",
        created_by: str = "",
        assignee: str = None,
        priority: str = "normal",
        tags=None,
        depends_on=None,
        estimate_minutes: int = None,
    ) -> dict:
        title = (title or "").strip()
        if not title:
            raise ValueError("title is required")
        if priority not in PRIORITIES:
            priority = "normal"
        with self.lock:
            now = _now()
            task = {
                "id": "task-" + uuid.uuid4().hex[:8],
                "title": _clean(title, 300),
                "detail": _clean(detail),
                "createdBy": created_by or "unknown",
                "assignee": assignee,
                "status": "todo",
                "priority": priority,
                "tags": [str(t)[:40] for t in (tags or [])][:20],
                "dependsOn": [str(d) for d in (depends_on or [])][:20],
                "estimateMinutes": int(estimate_minutes) if estimate_minutes else None,
                "created": now,
                "updated": now,
                "lease": None,
                "result": None,
                "artifacts": [],
                "history": [{"ts": now, "actor": created_by or "unknown", "action": "created"}],
            }
            self.tasks[task["id"]] = task
            self._emit_unlocked("task.created", {"taskId": task["id"], "title": task["title"], "priority": priority})
            self._save()
            return dict(task)

    def list_tasks(self, status: str = None, assignee: str = None, tag: str = None, include_done: bool = True) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            rows = list(self.tasks.values())
        if status:
            wanted = {s.strip() for s in status.split(",") if s.strip()}
            rows = [t for t in rows if t["status"] in wanted]
        if not include_done:
            rows = [t for t in rows if t["status"] not in ("done", "cancelled")]
        if assignee:
            rows = [t for t in rows if t.get("assignee") == assignee or t.get("createdBy") == assignee]
        if tag:
            rows = [t for t in rows if tag in (t.get("tags") or [])]
        rows.sort(key=lambda t: (
```

```
            {"in_progress": 0, "blocked": 1, "todo": 2, "done": 3, "cancelled": 4}.get(t["status"], 5),
            _PRIORITY_RANK.get(t.get("priority"), 2),
            t.get("created", 0),
        ))
        now = _now()
        out = []
        for t in rows:
            row = dict(t)
            lease = t.get("lease")
            row["leaseRemainingSeconds"] = round(lease["expiresAt"] - now, 1) if lease and lease.get("expiresAt") else None
            row["blockedBy"] = [d for d in (t.get("dependsOn") or []) if self.tasks.get(d, {}).get("status") != "done"]
            out.append(row)
        return {
            "tasks": out,
            "count": len(out),
            "counts": {
                s: sum(1 for t in self.tasks.values() if t["status"] == s) for s in TASK_STATUSES
            },
            "serverTime": now,
        }

    def _resolve_agent_unlocked(self, agent_id: str) -> str:
        if agent_id in self.agents:
            return agent_id
        match = next((a["id"] for a in self.agents.values() if a["name"].lower() == str(agent_id).lower()), None)
        if not match:
            raise KeyError(f"Unknown agent: {agent_id!r} - register first via action=register")
        return match

    def claim_task(self, task_id: str, agent_id: str, lease_seconds: int = None) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            agent_id = self._resolve_agent_unlocked(agent_id)
            if task["status"] in ("done", "cancelled"):
                raise ValueError(f"Task {task_id} is already {task['status']}")
            blocked = [d for d in (task.get("dependsOn") or []) if self.tasks.get(d, {}).get("status") != "done"]
            if blocked:
                raise ValueError(f"Task {task_id} is blocked by incomplete dependencies: {', '.join(blocked)}")
            current = task.get("assignee")
            if current and current != agent_id and task.get("lease"):
                raise ValueError(f"Task {task_id} is already claimed by {current} until the lease expires")
            now = _now()
            lease_s = int(lease_seconds or self.default_lease_seconds)
            task.update({
                "assignee": agent_id,
                "status": "in_progress",
                "updated": now,
                "lease": {"owner": agent_id, "claimedAt": now, "expiresAt": now + lease_s, "seconds": lease_s},
            })
            task.setdefault("history", []).append({"ts": now, "actor": agent_id, "action": "claimed"})
            self._emit_unlocked("task.claimed", {"taskId": task_id, "agentId": agent_id, "leaseSeconds": lease_s})
            self._save()
            return dict(task)

    def renew_task(self, task_id: str, agent_id: str, lease_seconds: int = None) -> dict:
        with self.lock:
            task = self.tasks.get(task_id)
```

```
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            lease = task.get("lease") or {}
            if lease.get("owner") != agent_id:
                raise ValueError("Only the current lease owner can renew this task")
            lease_s = int(lease_seconds or lease.get("seconds") or self.default_lease_seconds)
            lease["expiresAt"] = _now() + lease_s
            task["lease"] = lease
            task["updated"] = _now()
            self._save()
            return dict(task)

    def release_task(self, task_id: str, agent_id: str, note: str = "") -> dict:
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            task.update({"assignee": None, "status": "todo", "lease": None, "updated": _now()})
            task.setdefault("history", []).append({"ts": _now(), "actor": agent_id, "action": "released", "note": note})
            self._emit_unlocked("task.released", {"taskId": task_id, "agentId": agent_id})
            self._save()
            return dict(task)

    def update_task(self, task_id: str, actor: str = "unknown", **changes) -> dict:
        allowed = {"status", "priority", "assignee", "detail", "tags", "dependsOn", "title"}
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            if changes.get("status") and changes["status"] not in TASK_STATUSES:
                raise ValueError(f"status must be one of {', '.join(TASK_STATUSES)}")
            if changes.get("priority") and changes["priority"] not in PRIORITIES:
                raise ValueError(f"priority must be one of {', '.join(PRIORITIES)}")
            for key, value in changes.items():
                if key in allowed and value is not None:
                    if key in ("detail", "title"):
                        value = _clean(value, 300 if key == "title" else 8000)
                    task[key] = value
            task["updated"] = _now()
            task.setdefault("history", []).append({"ts": _now(), "actor": actor, "action": "updated", "changes": sorted(changes)})
            self._emit_unlocked("task.updated", {"taskId": task_id, "actor": actor, "changes": sorted(changes)})
            self._save()
            return dict(task)

    def complete_task(self, task_id: str, agent_id: str, result: str = "", artifacts=None, status: str = "done") -> dict:
        if status not in ("done", "cancelled"):
            raise ValueError("complete status must be 'done' or 'cancelled'")
        with self.lock:
            task = self.tasks.get(task_id)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            lease_owner = (task.get("lease") or {}).get("owner")
            if lease_owner and lease_owner != agent_id and task.get("assignee") not in (None, agent_id):
                raise ValueError(f"Task {task_id} is leased to {lease_owner}")
            now = _now()
            task.update({
                "status": status,
                "assignee": task.get("assignee") or agent_id,
                "result": _clean(result),
                "artifacts": [str(a)[:500] for a in (artifacts or [])][:50],
```

```
                "lease": None,
                "updated": now,
                "completedAt": now,
            })
            task.setdefault("history", []).append({"ts": now, "actor": agent_id, "action": status})
            self._emit_unlocked(f"task.{status}", {"taskId": task_id, "agentId": agent_id, "title": task["title"]})
            self._save()
            return dict(task)

    def delete_task(self, task_id: str, actor: str = "unknown") -> dict:
        with self.lock:
            task = self.tasks.pop(task_id, None)
            if not task:
                raise KeyError(f"Unknown task id: {task_id!r}")
            self._emit_unlocked("task.deleted", {"taskId": task_id, "actor": actor})
            self._save()
            return {"deleted": task_id}

    # -- feed / export ---------------------------------------------------

    def feed(self, since: int = 0, limit: int = 100) -> dict:
        limit = max(1, min(int(limit or 100), 500))
        with self.lock:
            rows = [e for e in self.events if e["seq"] > since][-limit:]
            seq = self.seq
        return {"events": rows, "count": len(rows), "cursor": seq}

    def export_handoff(self, path: str = None, include_done: bool = True) -> dict:
        """Render a markdown brief that any next agent (or human) can pick up."""
        with self.lock:
            agents = [self._agent_view_unlocked(a) for a in self.agents.values()]
            board = self.list_tasks(include_done=include_done)
            messages = list(self.messages)[-25:]
            stats = self.stats()

        lines = [
            "# Agent handoff brief",
            "",
            f"Generated: {time.strftime('%Y-%m-%d %H:%M:%S')}  ·  bus v{BUS_VERSION}",
            "",
            "## Roster",
            "",
        ]
        if agents:
            lines += ["| Agent | Role | Status | Open tasks | Capabilities |", "|---|---|---|---|---|"]
            for a in agents:
                lines.append(
                    f"| {a['name']} (`{a['id']}`) | {a['role']} | {a['status']} | {a['openTasks']} | "
                    f"{', '.join(a.get('capabilities') or []) or '-'} |"
                )
        else:
            lines.append("_No agents registered yet._")

        lines += ["", "## Task board", ""]
        if board["tasks"]:
            for t in board["tasks"]:
                mark = {"done": "x"}.get(t["status"], " ")
                owner = f" - @{t['assignee']}" if t.get("assignee") else ""
                lines.append(f"- [{mark}] **{t['title']}** ({t['status']}, {t['priority']}){owner}  ")
                if t.get("detail"):
```

```
                    lines.append(f"  {t['detail']}")
                if t.get("result"):
                    lines.append(f"  _Result:_ {t['result']}")
        else:
            lines.append("_Task board is empty._")

        lines += ["", "## Recent messages", ""]
        if messages:
            for m in messages:
                target = m.get("to") or "broadcast"
                lines.append(
                    f"- **{m['from']} -> {target}** [{m['kind']}/{m['priority']}] "
                    f"{m.get('subject') or '(no subject)'}: {m.get('body', '')[:400]}"
                )
        else:
            lines.append("_No messages._")

        lines += [
            "",
            "## Live bus stats",
            "",
            f"- agents: {stats['agents']} ({stats['agentsOnline']} online)",
            f"- tasks: {stats['tasks']} total · " + ", ".join(f"{k}={v}" for k, v in stats["taskCounts"].items()),
            f"- messages: {stats['messages']}",
            "",
            "Pick a task with `agent_bus {action:'task_claim', taskId:'...', agentId:'...'}`, "
            "or post status with `agent_bus {action:'post', ...}`.",
            "",
        ]
        text = "\n".join(lines)
        target = None
        if path:
            target = Path(path)
            target.parent.mkdir(parents=True, exist_ok=True)
            target.write_text(text, encoding="utf-8")
        return {"markdown": text, "path": str(target) if target else None, "bytes": len(text.encode("utf-8"))}

    def stats(self) -> dict:
        with self.lock:
            self._expire_leases_unlocked()
            agents = [self._agent_view_unlocked(a) for a in self.agents.values()]
            return {
                "version": BUS_VERSION,
                "agents": len(agents),
                "agentsOnline": sum(1 for a in agents if a["online"]),
                "tasks": len(self.tasks),
                "taskCounts": {s: sum(1 for t in self.tasks.values() if t["status"] == s) for s in TASK_STATUSES},
                "messages": len(self.messages),
                "events": len(self.events),
                "cursor": self.seq,
                "unreadTotal": sum(a["inboxUnread"] for a in agents),
                "serverTime": _now(),
            }

    # -- single entry point used by the HTTP + MCP layers ---------------------

    def handle(self, payload: dict) -> dict:
        action = (payload or {}).get("action", "stats")
        p = payload or {}
        if action == "register":
```

```
            return self.register_agent(
                name=p.get("name") or p.get("agent") or "agent",
                role=p.get("role") or "agent",
                capabilities=p.get("capabilities"),
                agent_id=p.get("agentId"),
                ttl=p.get("ttl"),
                meta=p.get("meta"),
            )
        if action == "heartbeat":
            return self.heartbeat(p.get("agentId") or p.get("agent"), p.get("status"))
        if action == "status":
            if p.get("agentId") and p.get("status"):
                return self.set_status(p["agentId"], p["status"])
            return self.stats()
        if action in ("agents", "list_agents"):
            return self.list_agents(bool(p.get("includeOffline", True)))
        if action in ("post", "send"):
            return self.post_message(
                sender=p.get("agentId") or p.get("from") or p.get("agent"),
                body=p.get("body") or p.get("message"),
                to=p.get("to"),
                subject=p.get("subject"),
                kind=p.get("kind") or "note",
                priority=p.get("priority") or "normal",
                thread_id=p.get("threadId"),
                meta=p.get("meta"),
            )
        if action == "inbox":
            return self.inbox(
                p.get("agentId") or p.get("agent"),
                since=int(p.get("since") or 0),
                limit=int(p.get("limit") or 50),
                unread_only=bool(p.get("unreadOnly")),
                include_broadcast=p.get("includeBroadcast", True),
            )
        if action == "thread":
            return self.read_thread(p.get("threadId") or p.get("id"))
        if action == "mark_read":
            return self.mark_read(
                p.get("agentId") or p.get("agent"),
                p.get("messageIds"),
                bool(p.get("all")),
            )
        if action in ("tasks", "list_tasks"):
            return self.list_tasks(
                p.get("status"), p.get("assignee"), p.get("tag"),
                include_done=p.get("includeDone", True),
            )
        if action == "task_create":
            return self.create_task(
                title=p.get("title"),
                detail=p.get("detail"),
                created_by=p.get("agentId") or p.get("createdBy") or "unknown",
                assignee=p.get("assignee"),
                priority=p.get("priority") or "normal",
                tags=p.get("tags"),
                depends_on=p.get("dependsOn"),
                estimate_minutes=p.get("estimateMinutes"),
            )
        if action == "task_claim":
```

```
            return self.claim_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("leaseSeconds"))
        if action == "task_renew":
            return self.renew_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("leaseSeconds"))
        if action == "task_release":
            return self.release_task(p.get("taskId") or p.get("id"), p.get("agentId") or p.get("agent"), p.get("note") or "")
        if action == "task_update":
            return self.update_task(
                p.get("taskId") or p.get("id"),
                actor=p.get("agentId") or p.get("actor") or "unknown",
                **{k: p.get(k) for k in ("status", "priority", "assignee", "detail", "tags", "dependsOn", "title") if k in p},
            )
        if action == "task_complete":
            return self.complete_task(
                p.get("taskId") or p.get("id"),
                p.get("agentId") or p.get("agent"),
                result=p.get("result") or "",
                artifacts=p.get("artifacts"),
                status=p.get("status") or "done",
            )
        if action == "task_delete":
            return self.delete_task(p.get("taskId") or p.get("id"), p.get("actor") or "unknown")
        if action in ("feed", "events"):
            return self.feed(int(p.get("since") or 0), int(p.get("limit") or 100))
        if action in ("export", "handoff"):
            return self.export_handoff(p.get("path"), bool(p.get("includeDone", True)))
        if action == "stats":
            return self.stats()
        raise ValueError(
            "Unknown agent_bus action. Use one of: register, heartbeat, status, agents, post, inbox, "
            "thread, mark_read, tasks, task_create, task_claim, task_renew, task_release, task_update, "
            "task_complete, task_delete, feed, export, stats."
        )

```

roblox-bridge/auth_core.py

273 lines

Scoped tokens, expiry, revocation, token-bucket limits

```python
#!/usr/bin/env python3
"""
auth_core.py - Scoped access tokens and per-token rate limiting for the bridge.

The single `bridge.token` you already have keeps working unchanged: it is treated
as the full-privilege owner credential. On top of that you can mint named,
expiring, scoped tokens so that:

  * a read-only inspector agent cannot mutate a place,
  * a teammate's CI can push scripts but cannot mint tokens or start tunnels,
  * a demo viewer can watch telemetry without touching Studio,
  * a runaway agent gets throttled instead of flooding the job queue.

Only token *hashes* are stored, and only the first 8 characters of a token are
kept for display. Tokens are shown in full exactly once, when minted.

Zero dependencies.
"""

import hashlib
import json
import os
import secrets
import threading
import time
from pathlib import Path

AUTH_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

ROLE_RANK = {"read": 1, "write": 2, "admin": 3}
ROLE_DESCRIPTIONS = {
    "read": "Inspect only: state, peers, survey, script reads, telemetry.",
    "write": "Everything read can do, plus Studio mutations, jobs and bus writes.",
    "admin": "Full control: token management, tunnel control, queue clearing, audit.",
}

# Paths that require a specific role regardless of the HTTP method.
READ_ONLY_PATHS = {
    "/api/health", "/api/state", "/api/peers", "/api/jobs", "/api/agent-guide",
    "/api/metrics", "/api/audit", "/api/events", "/api/ops", "/api/self-test",
}
ADMIN_PATHS = {"/api/tokens", "/api/audit/verify"}


def _now() -> float:
    return time.time()


def _hash(raw: str) -> str:
    return hashlib.sha256(raw.encode("utf-8")).hexdigest()


class TokenRecord:
    __slots__ = ("id", "label", "role", "created", "expires", "revoked", "last_used",
                 "uses", "rate_limit", "burst", "bucket", "bucket_ts", "token_hash", "prefix")

    def __init__(self, **kw):
        for slot in self.__slots__:
            setattr(self, slot, kw.get(slot))
```

```
61
62      def view(self) -> dict:
63          return {
64              "id": self.id,
65              "label": self.label,
66              "role": self.role,
67              "prefix": self.prefix,
68              "created": self.created,
69              "expires": self.expires,
70              "revoked": bool(self.revoked),
71              "lastUsed": self.last_used,
72              "uses": self.uses,
73              "rateLimitPerSecond": self.rate_limit,
74              "burst": self.burst,
75          }
76
77      def active(self, now: float = None) -> bool:
78          now = now or _now()
79          if self.revoked:
80              return False
81          if self.expires and self.expires < now:
82              return False
83          return True
84
85
86  class TokenStore:
87      """Hashed token registry with role scopes, expiry, revocation and rate limits."""
88
89      def __init__(self, path=None, primary_token: str = "", persist: bool = True, default_rate: float = 20.0):
90          self.path = Path(path) if path else DEFAULT_STATE_DIR / "tokens.json"
91          self.persist = bool(persist)
92          self.lock = threading.RLock()
93          self.default_rate = float(default_rate)
94          self.records = {}       # token id -> TokenRecord
95          self.by_hash = {}       # sha256(token) -> token id
96          self.primary = None     # TokenRecord for the bridge.token owner credential
97          if primary_token:
98              self.primary = TokenRecord(
99                  id="primary", label="owner (bridge.token)", role="admin",
100                 created=_now(), expires=None, revoked=False, last_used=None, uses=0,
101                 rate_limit=0.0, burst=0, bucket=0.0, bucket_ts=_now(),
102                 token_hash=_hash(primary_token), prefix=primary_token[:8],
103             )
104             self.by_hash[self.primary.token_hash] = "primary"
105         self._load()
106
107     # -- persistence ------------------------------------------------------
108
109     def _load(self):
110         if not self.persist or not self.path.is_file():
111             return
112         try:
113             raw = json.loads(self.path.read_text(encoding="utf-8"))
114         except (OSError, ValueError):
115             return
116         for row in raw.get("tokens", []):
117             try:
118                 rec = TokenRecord(
119                     id=row["id"], label=row.get("label") or "", role=row.get("role") or "read",
120                     created=row.get("created") or _now(), expires=row.get("expires"),
```

```
                    revoked=bool(row.get("revoked")), last_used=row.get("lastUsed"), uses=int(row.get("uses") or 0),
                    rate_limit=float(row.get("rateLimitPerSecond") or 0.0), burst=float(row.get("burst") or 0.0),
                    bucket=float(row.get("burst") or 0.0), bucket_ts=_now(),
                    token_hash=row["hash"], prefix=row.get("prefix") or "",
                )
            except (KeyError, TypeError, ValueError):
                continue
            self.records[rec.id] = rec
            self.by_hash[rec.token_hash] = rec.id

    def _save(self):
        if not self.persist:
            return
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            payload = {
                "version": AUTH_VERSION,
                "saved": _now(),
                "tokens": [
                    dict(r.view(), hash=r.token_hash)
                    for r in self.records.values()
                ],
            }
            tmp = self.path.with_suffix(".tmp")
            tmp.write_text(json.dumps(payload, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
            tmp.chmod(0o600)
            os.replace(tmp, self.path)
        except OSError:
            pass

    # -- authentication ------------------------------------------------------

    def authenticate(self, raw: str, consume_rate: bool = True):
        """Return (record, rate_info) or (None, None) when the token is invalid."""
        if not raw:
            return None, None
        token_hash = _hash(raw)
        with self.lock:
            rec_id = self.by_hash.get(token_hash)
            if not rec_id:
                return None, None
            rec = self.primary if rec_id == "primary" else self.records.get(rec_id)
            if not rec or not rec.active():
                return None, None
            rate_info = self._rate_check_unlocked(rec) if consume_rate else {"allowed": True, "retryAfter": 0}
            if not rate_info["allowed"]:
                return rec, rate_info
            rec.last_used = _now()
            rec.uses = int(rec.uses or 0) + 1
            return rec, rate_info

    def _rate_check_unlocked(self, rec: TokenRecord) -> dict:
        rate = float(rec.rate_limit or 0.0)
        burst = float(rec.burst or rate * 2 or 0.0)
        if rate <= 0:
            return {"allowed": True, "retryAfter": 0, "remaining": None, "limit": 0}
        now = _now()
        elapsed = max(0.0, now - float(rec.bucket_ts or now))
        tokens = min(burst, float(rec.bucket or 0.0) + elapsed * rate)
        if tokens < 1.0:
```

```
            deficit = 1.0 - tokens
            retry = max(1, int(deficit / rate) + 1)
            rec.bucket = tokens
            rec.bucket_ts = now
            return {"allowed": False, "retryAfter": retry, "remaining": 0, "limit": rate}
        rec.bucket = tokens - 1.0
        rec.bucket_ts = now
        return {"allowed": True, "retryAfter": 0, "remaining": int(rec.bucket), "limit": rate}

    # -- management ------------------------------------------------------------

    def create(self, label: str, role: str = "write", ttl_seconds: int = None, rate_limit: float = None, burst: float = None) -> dict:
        role = (role or "write").lower()
        if role not in ROLE_RANK:
            raise ValueError(f"role must be one of: {', '.join(ROLE_RANK)}")
        raw = "arb_" + secrets.token_urlsafe(30)
        rate = self.default_rate if rate_limit is None else float(rate_limit)
        with self.lock:
            rec = TokenRecord(
                id="tok-" + secrets.token_hex(6),
                label=(label or "unnamed")[:120],
                role=role,
                created=_now(),
                expires=(_now() + int(ttl_seconds)) if ttl_seconds else None,
                revoked=False,
                last_used=None,
                uses=0,
                rate_limit=rate,
                burst=float(burst if burst is not None else max(rate * 2, 1.0)),
                bucket=float(burst if burst is not None else max(rate * 2, 1.0)),
                bucket_ts=_now(),
                token_hash=_hash(raw),
                prefix=raw[:8],
            )
            self.records[rec.id] = rec
            self.by_hash[rec.token_hash] = rec.id
            self._save()
            view = rec.view()
            view["token"] = raw
            view["note"] = "Store this token now - only its hash and prefix are retained."
            return view

    def revoke(self, token_id: str) -> dict:
        with self.lock:
            if token_id == "primary":
                raise ValueError("The owner credential (bridge.token) cannot be revoked here; rotate it instead.")
            rec = self.records.get(token_id)
            if not rec:
                raise KeyError(f"Unknown token id: {token_id!r}")
            rec.revoked = True
            self._save()
            return rec.view()

    def list(self) -> dict:
        with self.lock:
            rows = []
            if self.primary:
                rows.append(self.primary.view())
            rows += [r.view() for r in self.records.values()]
        rows.sort(key=lambda r: (r["revoked"], r["created"] or 0))
```

```
        return {
            "tokens": rows,
            "count": len(rows),
            "active": sum(1 for r in rows if not r["revoked"] and (not r["expires"] or r["expires"] > _now())),
            "roles": ROLE_DESCRIPTIONS,
        }

    def owner_token(self) -> str:
        """Only used by tests/first-run bootstrap; never exposed over HTTP."""
        return ""

    @staticmethod
    def role_allows(role: str, required: str) -> bool:
        return ROLE_RANK.get(role or "read", 0) >= ROLE_RANK.get(required or "read", 0)

    def handle(self, payload: dict) -> dict:
        action = (payload or {}).get("action", "list")
        p = payload or {}
        if action == "list":
            return self.list()
        if action == "create":
            return self.create(
                p.get("label") or "unnamed",
                p.get("role") or "write",
                p.get("ttlSeconds"),
                p.get("rateLimitPerSecond"),
                p.get("burst"),
            )
        if action == "revoke":
            return self.revoke(p.get("id") or p.get("tokenId"))
        if action == "roles":
            return {"roles": ROLE_DESCRIPTIONS, "rank": ROLE_RANK}
        raise ValueError("Unknown token action. Use list, create, revoke or roles.")

```

roblox-bridge/telemetry.py

471 lines

Metrics, SSE event stream, hash-chained audit log

```python
#!/usr/bin/env python3
"""
telemetry.py - Metrics, live event stream, and tamper-evident audit log for the
Arena Roblox Studio bridge.

Zero-dependency. Three cooperating pieces:

  Metrics      counters / gauges / latency histograms with label sets, plus a
               Prometheus text exposition and a JSON snapshot for the dashboard.
  EventStream  bounded, sequence-numbered event ring with condition-variable
               waits, powering `GET /api/events` (Server-Sent Events).
  AuditLog     append-only JSONL log where every record chains the SHA-256 hash
               of its predecessor, so silent edits/tampering are detectable with
               `verify()`.

All three are thread-safe and safe to embed in server.py.
"""

import hashlib
import json
import os
import threading
import time
from collections import deque
from pathlib import Path

TELEMETRY_VERSION = "5.0.0"
DEFAULT_STATE_DIR = Path(__file__).resolve().parent / "bridge_state"

# Bounded sample count per histogram key; percentiles are computed from this
# reservoir and the true count/sum/min/max are kept exactly.
_HIST_SAMPLES = 2048


def _now() -> float:
    return time.time()


def _file_lock(fh):
    """Best-effort advisory exclusive lock for cross-process audit appends."""
    try:
        import fcntl
        fcntl.flock(fh.fileno(), fcntl.LOCK_EX)
        return "fcntl"
    except (ImportError, OSError):
        try:
            import msvcrt
            fh.seek(0)
            msvcrt.locking(fh.fileno(), msvcrt.LK_LOCK, 1)
            return "msvcrt"
        except (ImportError, OSError):
            return None


def _file_unlock(fh, kind):
    try:
        if kind == "fcntl":
            import fcntl
            fcntl.flock(fh.fileno(), fcntl.LOCK_UN)
        elif kind == "msvcrt":
```

```
            import msvcrt
            fh.seek(0)
            msvcrt.locking(fh.fileno(), msvcrt.LK_UNLCK, 1)
    except (ImportError, OSError):
        pass


def _label_key(labels: dict) -> str:
    if not labels:
        return ""
    return ",".join(f"{k}={labels[k]}" for k in sorted(labels))


def _fmt_num(value) -> str:
    if isinstance(value, bool):
        return "1" if value else "0"
    if isinstance(value, int):
        return str(value)
    if isinstance(value, float):
        if value == int(value) and abs(value) < 1e15:
            return str(int(value))
        return f"{value:.6f}".rstrip("0").rstrip(".")
    return str(value)


class Metrics:
    """Counter/gauge/histogram registry with Prometheus + JSON output."""

    def __init__(self):
        self.lock = threading.Lock()
        self.started = _now()
        self.counters = {}   # key -> {"name","labels","value"}
        self.gauges = {}
        self.histograms = {}  # key -> {"name","labels","count","sum","min","max","samples":deque}
        self.histogram_order = {}

    def _inc_unlocked(self, name: str, labels: dict, value: float):
        key = (name, _label_key(labels))
        row = self.counters.get(key)
        if row is None:
            row = {"name": name, "labels": dict(labels or {}), "value": 0.0}
            self.counters[key] = row
        row["value"] += value
        return row

    def inc(self, name: str, value: float = 1, labels: dict = None):
        with self.lock:
            self._inc_unlocked(name, labels or {}, float(value))

    def gauge(self, name: str, value: float, labels: dict = None):
        with self.lock:
            key = (name, _label_key(labels or {}))
            self.gauges[key] = {"name": name, "labels": dict(labels or {}), "value": float(value)}

    def observe(self, name: str, value: float, labels: dict = None, buckets=None):
        value = float(value)
        with self.lock:
            key = (name, _label_key(labels or {}))
            row = self.histograms.get(key)
            if row is None:
```

```
                row = {
                    "name": name,
                    "labels": dict(labels or {}),
                    "count": 0,
                    "sum": 0.0,
                    "min": value,
                    "max": value,
                    "samples": deque(maxlen=_HIST_SAMPLES),
                    "buckets": list(buckets or ()),
                    "bucketCounts": {b: 0 for b in (buckets or ())},
                }
                self.histograms[key] = row
            row["count"] += 1
            row["sum"] += value
            row["min"] = min(row["min"], value)
            row["max"] = max(row["max"], value)
            row["samples"].append(value)
            for b in row["buckets"]:
                if value <= b:
                    row["bucketCounts"][b] += 1
            self.histogram_order[key] = self.histogram_order.pop(
                key, len(self.histogram_order)
            )
            return row

    @staticmethod
    def _percentile(sorted_samples, p: float) -> float:
        if not sorted_samples:
            return 0.0
        if len(sorted_samples) == 1:
            return float(sorted_samples[0])
        rank = (len(sorted_samples) - 1) * p
        lo = int(rank)
        hi = min(lo + 1, len(sorted_samples) - 1)
        frac = rank - lo
        return float(sorted_samples[lo] * (1 - frac) + sorted_samples[hi] * frac)

    def summarize_histogram(self, row: dict) -> dict:
        samples = sorted(row["samples"])
        return {
            "count": row["count"],
            "sum": round(row["sum"], 3),
            "min": round(row["min"], 3),
            "max": round(row["max"], 3),
            "mean": round(row["sum"] / row["count"], 3) if row["count"] else 0.0,
            "p50": round(self._percentile(samples, 0.50), 3),
            "p90": round(self._percentile(samples, 0.90), 3),
            "p95": round(self._percentile(samples, 0.95), 3),
            "p99": round(self._percentile(samples, 0.99), 3),
        }

    def snapshot(self, uptime: float = None) -> dict:
        with self.lock:
            counters = [dict(v) for v in self.counters.values()]
            gauges = [dict(v) for v in self.gauges.values()]
            histograms = []
            for row in self.histograms.values():
                entry = {"name": row["name"], "labels": row["labels"]}
                entry.update(self.summarize_histogram(row))
                histograms.append(entry)
```

```
        counters.sort(key=lambda r: (r["name"], _label_key(r["labels"])))
        gauges.sort(key=lambda r: (r["name"], _label_key(r["labels"])))
        histograms.sort(key=lambda r: r["name"])
        return {
            "version": TELEMETRY_VERSION,
            "uptimeSeconds": round(uptime if uptime is not None else (_now() - self.started), 2),
            "counters": counters,
            "gauges": gauges,
            "histograms": histograms,
        }

    def prometheus(self, extra_gauges: dict = None) -> str:
        lines = []
        with self.lock:
            counters = list(self.counters.values())
            gauges = list(self.gauges.values())
            histograms = list(self.histograms.values())
        seen_types = set()

        def _decl(name, kind):
            if name not in seen_types:
                seen_types.add(name)
                lines.append(f"# TYPE {name} {kind}")

        for row in counters:
            _decl(row["name"], "counter")
            lines.append(f"{row['name']}{self._fmt_labels(row['labels'])} {_fmt_num(row['value'])}")
        for row in gauges:
            _decl(row["name"], "gauge")
            lines.append(f"{row['name']}{self._fmt_labels(row['labels'])} {_fmt_num(row['value'])}")
        for row in histograms:
            name = row["name"]
            _decl(name, "histogram")
            labels = dict(row["labels"])
            total = row["count"] or 1
            for b, count in row["bucketCounts"].items():
                bucket_labels = dict(labels)
                bucket_labels["le"] = b
                lines.append(f"{name}_bucket{self._fmt_labels(bucket_labels)} {count}")
            bucket_labels = dict(labels)
            bucket_labels["le"] = "+Inf"
            lines.append(f"{name}_bucket{self._fmt_labels(bucket_labels)} {row['count']}")
            summary = self.summarize_histogram(row)
            lines.append(f"{name}_sum{self._fmt_labels(labels)} {_fmt_num(round(row['sum'], 3))}")
            lines.append(f"{name}_count{self._fmt_labels(labels)} {row['count']}")
            for q, key in (("0.5", "p50"), ("0.9", "p90"), ("0.95", "p95"), ("0.99", "p99")):
                q_labels = dict(labels)
                q_labels["quantile"] = q
                lines.append(f"{name}{self._fmt_labels(q_labels)} {_fmt_num(summary[key])}")
            _ = total
        for name, value in (extra_gauges or {}).items():
            try:
                _decl(name, "gauge")
                lines.append(f"{name} {_fmt_num(float(value))}")
            except (TypeError, ValueError):
                continue
        return "\n".join(lines) + "\n"

    @staticmethod
    def _fmt_labels(labels: dict) -> str:
```

```
        if not labels:
            return ""
        parts = []
        for key in sorted(labels):
            value = str(labels[key]).replace("\\", "\\\\").replace('"', '\\"').replace("\n", " ")
            parts.append(f'{key}="{value}"')
        return "{" + ",".join(parts) + "}"


class EventStream:
    """Sequence-numbered event ring with blocking tail reads (SSE friendly)."""

    def __init__(self, max_events: int = 1500):
        self.lock = threading.Lock()
        self.cv = threading.Condition(self.lock)
        self.events = deque(maxlen=max_events)
        self.seq = 0

    def emit(self, kind: str, data: dict = None, level: str = "info") -> dict:
        with self.cv:
            self.seq += 1
            event = {
                "seq": self.seq,
                "ts": _now(),
                "kind": kind,
                "level": level,
                "data": data or {},
            }
            self.events.append(event)
            self.cv.notify_all()
            return event

    def since(self, since: int = 0, limit: int = 200, kinds=None) -> dict:
        limit = max(1, min(int(limit or 200), 1000))
        with self.lock:
            rows = [e for e in self.events if e["seq"] > since]
            if kinds:
                wanted = {k.strip() for k in str(kinds).split(",") if k.strip()}
                rows = [e for e in rows if any(e["kind"].startswith(w) for w in wanted)]
            rows = rows[-limit:]
            cursor = self.seq
        return {"events": rows, "count": len(rows), "cursor": cursor}

    def tail(self, cursor: int, timeout: float = 15.0, limit: int = 100) -> dict:
        deadline = time.monotonic() + max(0.0, timeout)
        with self.cv:
            while self.seq <= cursor:
                remaining = deadline - time.monotonic()
                if remaining <= 0:
                    return {"events": [], "count": 0, "cursor": self.seq, "timeout": True}
                self.cv.wait(timeout=min(remaining, 1.0))
            rows = [e for e in self.events if e["seq"] > cursor][:limit]
            return {"events": rows, "count": len(rows), "cursor": self.seq, "timeout": False}


class AuditLog:
    """Append-only, hash-chained audit log (JSONL)."""

    GENESIS = "0" * 64

```

```
    def __init__(self, path=None, persist: bool = True, max_memory: int = 5000):
        self.path = Path(path) if path else DEFAULT_STATE_DIR / "audit.jsonl"
        self.persist = bool(persist)
        self.lock = threading.Lock()
        self.entries = deque(maxlen=max_memory)
        self.last_hash = self.GENESIS
        self.seq = 0
        self._load()

    def _load(self):
        if not self.persist or not self.path.is_file():
            return
        try:
            with self.path.open("r", encoding="utf-8") as fh:
                for line in fh:
                    line = line.strip()
                    if not line:
                        continue
                    try:
                        row = json.loads(line)
                    except ValueError:
                        continue
                    self.entries.append(row)
                    self.last_hash = row.get("hash") or self.last_hash
                    self.seq = max(self.seq, int(row.get("seq") or 0))
        except OSError:
            pass

    def _read_disk_tail(self, fh) -> dict:
        """Read the last complete JSONL record from an open file handle."""
        try:
            fh.seek(0, os.SEEK_END)
            size = fh.tell()
            window = min(size, 16384)
            fh.seek(size - window)
            chunk = fh.read()
            lines = [ln for ln in chunk.splitlines() if ln.strip()]
            if not lines:
                return {}
            return json.loads(lines[-1])
        except (OSError, ValueError):
            return {}

    def _sync_head_from_disk(self):
        """Adopt a newer on-disk head so concurrent writers keep one chain."""
        try:
            with self.path.open("r", encoding="utf-8") as fh:
                row = self._read_disk_tail(fh)
        except OSError:
            return
        head = row.get("hash")
        if head and head != self.last_hash:
            self.last_hash = head
            self.seq = max(self.seq, int(row.get("seq") or 0))

    def append(self, action: str, actor: str = "system", target: str = "", detail=None, ip: str = "", outcome: str = "ok") -> dict:
        with self.lock:
            if not self.persist:
                self._append_unlocked(action, actor, target, detail, ip, outcome)
                return dict(self.entries[-1])
```

```
            self.path.parent.mkdir(parents=True, exist_ok=True)
            try:
                with self.path.open("a+", encoding="utf-8") as fh:
                    lock = _file_lock(fh)
                    try:
                        self._sync_head_from_disk()
                        entry = self._append_unlocked(action, actor, target, detail, ip, outcome)
                        fh.write(json.dumps(entry, ensure_ascii=False, separators=(",", ":")) + "\n")
                        fh.flush()
                    finally:
                        _file_unlock(fh, lock)
                return dict(entry)
            except OSError:
                entry = self._append_unlocked(action, actor, target, detail, ip, outcome)
                return dict(entry)

    def _append_unlocked(self, action, actor, target, detail, ip, outcome) -> dict:
        self.seq += 1
        entry = {
            "seq": self.seq,
            "ts": _now(),
            "action": action,
            "actor": actor or "system",
            "target": target or "",
            "outcome": outcome,
            "ip": ip or "",
            "detail": detail if isinstance(detail, (dict, list, str, int, float, type(None))) else str(detail),
            "prev": self.last_hash,
        }
        canonical = json.dumps(entry, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
        entry["hash"] = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
        self.last_hash = entry["hash"]
        self.entries.append(entry)
        return entry

    def tail(self, limit: int = 100, action: str = None) -> dict:
        with self.lock:
            rows = list(self.entries)
        if action:
            rows = [r for r in rows if action in str(r.get("action", ""))]
        rows = rows[-max(1, min(int(limit or 100), 2000)):]
        rows.reverse()
        return {"entries": rows, "count": len(rows), "cursor": self.seq}

    def verify(self) -> dict:
        """Recompute the whole chain. Detects edit, reorder, deletion or insertion."""
        with self.lock:
            rows = list(self.entries)
        prev = self.GENESIS
        for index, row in enumerate(rows):
            stored = row.get("hash")
            body = {k: v for k, v in row.items() if k != "hash"}
            canonical = json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
            expected = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
            if row.get("prev") != prev:
                return {
                    "ok": False,
                    "checked": index + 1,
                    "brokenAt": row.get("seq"),
                    "reason": "prev-hash mismatch (entry inserted, removed or reordered)",
```

```
                }
            if expected != stored:
                return {
                    "ok": False,
                    "checked": index + 1,
                    "brokenAt": row.get("seq"),
                    "reason": "content hash mismatch (entry was edited)",
                }
            prev = stored
        return {
            "ok": True,
            "checked": len(rows),
            "head": prev,
            "entries": len(rows),
            "file": str(self.path) if self.persist else None,
        }


class Telemetry:
    """Convenience container wired into BridgeState."""

    def __init__(self, state_dir=None, persist: bool = True):
        state_dir = Path(state_dir) if state_dir else DEFAULT_STATE_DIR
        self.metrics = Metrics()
        self.events = EventStream()
        self.audit = AuditLog(state_dir / "audit.jsonl", persist=persist)
        self.started = _now()

    @property
    def uptime(self) -> float:
        return _now() - self.started

    def record_job(self, job: dict):
        """Called when a job finishes: counters + latency histogram + event."""
        ok = bool(job.get("ok"))
        labels = {"type": job.get("type") or "unknown", "dm": job.get("dm") or job.get("target_dm") or "auto"}
        self.metrics.inc("bridge_jobs_total", 1, dict(labels, outcome="ok" if ok else "error"))
        duration = job.get("duration_ms")
        if isinstance(duration, (int, float)):
            self.metrics.observe("bridge_job_duration_ms", float(duration), labels)
        self.events.emit(
            "job.finish",
            {
                "id": job.get("id"),
                "type": job.get("type"),
                "status": job.get("status"),
                "durationMs": duration,
                "error": job.get("error"),
            },
            level="info" if ok else "error",
        )

```

roblox-bridge/tools/seed_agent_bus.py

129 lines

Idempotent Agent Bus seeding (online or offline)

```python
#!/usr/bin/env python3
"""
seed_agent_bus.py - Populate the live bridge Agent Bus with the partner work order.

Reads `handoff/bus_seed.json` (agents, tasks, messages) and posts it to a running
bridge over HTTP, or writes it into an offline state directory with `--offline`.

Usage:
    python3 tools/seed_agent_bus.py                      # uses BRIDGE_URL/BRIDGE_TOKEN
    python3 tools/seed_agent_bus.py --url http://127.0.0.1:8077 --token <token>
    python3 tools/seed_agent_bus.py --offline --state-dir ./bridge_state
    python3 tools/seed_agent_bus.py --dry-run

Idempotent: re-running matches agents by name and skips tasks whose title
already exists on the board.
"""

import argparse
import json
import os
import sys
import urllib.error
import urllib.request
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent.parent
DEFAULT_SEED = ROOT / "handoff" / "bus_seed.json"


def http(url: str, token: str, payload: dict, timeout: float = 20.0) -> dict:
    data = json.dumps(payload).encode("utf-8")
    headers = {"Content-Type": "application/json"}
    if token:
        headers["X-Arena-Token"] = token
    req = urllib.request.Request(url.rstrip("/") + "/api/bus", data=data, method="POST", headers=headers)
    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            return json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as exc:
        raise RuntimeError(f"HTTP {exc.code}: {exc.read().decode('utf-8', 'replace')[:300]}") from exc
    except urllib.error.URLError as exc:
        raise RuntimeError(f"Cannot reach bridge at {url}: {exc.reason}") from exc


def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--url", default=os.environ.get("BRIDGE_URL", "http://127.0.0.1:8077"))
    ap.add_argument("--token", default=os.environ.get("BRIDGE_TOKEN") or None)
    ap.add_argument("--seed", default=str(DEFAULT_SEED))
    ap.add_argument("--offline", action="store_true", help="Write directly to a local state dir (no server)")
    ap.add_argument("--state-dir", default=None, help="State dir for --offline (default roblox-bridge/bridge_state)")
    ap.add_argument("--dry-run", action="store_true")
    args = ap.parse_args()

    seed_path = Path(args.seed)
    seed = json.loads(seed_path.read_text(encoding="utf-8"))

    if not args.token:
        token_file = HERE.parent / "bridge.token"
```

```
        if token_file.is_file():
            args.token = token_file.read_text(encoding="utf-8").strip() or None

    if args.offline:
        sys.path.insert(0, str(HERE.parent))
        from bus_core import AgentBus

        bus = AgentBus(Path(args.state_dir) / "bus.json" if args.state_dir else None)
        call = lambda payload: bus.handle(payload)  # noqa: E731
        existing_agents = bus.list_agents()["agents"]
        existing_tasks = bus.list_tasks()["tasks"]
    else:
        call = lambda payload: http(args.url, args.token, payload)  # noqa: E731
        existing_agents = call({"action": "agents"}).get("agents", [])
        existing_tasks = call({"action": "tasks"}).get("tasks", [])

    if args.dry_run:
        print(json.dumps({"agents": seed.get("agents", []), "tasks": seed.get("tasks", []),
                          "messages": seed.get("messages", [])}, indent=2, ensure_ascii=False))
        return 0

    registered = {}
    for agent in seed.get("agents", []):
        row = call({
            "action": "register",
            "name": agent["name"],
            "role": agent.get("role", "agent"),
            "capabilities": agent.get("capabilities", []),
        })
        registered[agent["name"]] = row
        print(f"[agent] {row['name']:<18} {row['id']}  role={row['role']}")

    known_titles = {t.get("title") for t in existing_tasks}
    created = 0
    for task in seed.get("tasks", []):
        if task.get("title") in known_titles:
            print(f"[task ] skip (exists) {task['title']}")
            continue
        row = call({
            "action": "task_create",
            "title": task["title"],
            "detail": task.get("detail", ""),
            "agentId": "seed",
            "priority": task.get("priority", "normal"),
            "tags": task.get("tags", []),
        })
        created += 1
        print(f"[task ] {row['id']}  {row['priority']:<8} {row['title']}")

    posted = 0
    for message in (seed.get("messages", []) if (created or not existing_tasks) else []):
        sender = registered.get(message.get("from", ""), {"id": message.get("from", "seed")})
        row = call({
            "action": "post",
            "agentId": sender["id"],
            "subject": message.get("subject", ""),
            "body": message.get("body", ""),
            "kind": message.get("kind", "note"),
            "priority": message.get("priority", "normal"),
        })
```

```
        posted += 1
        print(f"[msg  ] {row['id']}  {row['subject']}")

    print(f"\nseeded {len(registered)} agents, {created} new tasks, {posted} messages")
    return 0


if __name__ == "__main__":
    sys.exit(main())

```

5. Diffs against the previous release

Unified diffs produced with `git diff c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524 -- <file>`. Lines prefixed + are added, - removed.

roblox-bridge/server.py (diff)

1 lines

HTTP bridge, auth scopes, ops endpoints, dashboard tab 7

```
(no diff found for roblox-bridge/server.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/mcp_core.py (diff)

1 lines

agent_bus + bridge_ops tools, 3 resources, client helpers

```
(no diff found for roblox-bridge/mcp_core.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/arena_studio.py (diff)

1 lines

ops / metrics / audit / tunnel / tokens / bus CLI

```
(no diff found for roblox-bridge/arena_studio.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/setup.py (diff)

1 lines

Supervised tunnel launch + URL watcher

```
(no diff found for roblox-bridge/setup.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/test_bridge_mcp.py (diff)

1 lines

v5 test coverage (scopes, bus, reaping, audit, tunnel)

```
(no diff found for roblox-bridge/test_bridge_mcp.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/mcp_server.py (diff)

1 lines

Version banner

```
(no diff found for roblox-bridge/mcp_server.py against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

roblox-bridge/ab.ps1 (diff)

1 lines

Version banner + ops examples

```
(no diff found for roblox-bridge/ab.ps1 against c56f9b7c1ef0f3cd6fe41e0676c871202ffa5524)

```

6. Verification snapshot

The v5 suite exercises the new surface end-to-end against the stateful mock Studio and a fake tunnel binary. Live Studio acceptance (real peer + real public tunnel) is still open and is tracked as G7/G8 in the task PDF.

```
16b9ea7 handoff: PDF pack (v5 code + Gemini 4 Argon task briefs)
 SESSION.md                                |   11 +-
 handoff/GEMINI-4-ARGON.md                 |   72 +-
 handoff/Gemini-4-Argon-Tasks.pdf          |  290 +++++
 handoff/Tunneling-v5-Code.pdf             | 1747 ++++++++++++++++++++++++++++++
 roblox-bridge/CHANGELOG.md                |    1 +
 roblox-bridge/docs/OPS.md                 |   12 +-
 roblox-bridge/tools/build_handoff_pdfs.py | 1267 ++++++++++++++++++++++
 7 files changed, 3364 insertions(+), 36 deletions(-)
```